РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА
И ГОСУДАРСТВЕННОЙ СЛУЖБЫ
ПРИ ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ

КОМПЛЕКСНАЯ ПРАКТИЧЕСКАЯ РАБОТА

по дисциплине «Программная инженерия»

Вариант 13

Анализатор утилизации и эффективности автопарка

Fleet Utilization Evaluator

Выполнил: Латыйпов Амир Русланович
Группа: ОБ-230766-24
Преподаватель: Марухленко А. Л.

2026

СОДЕРЖАНИЕ

Введение ⋯ 3

Глава 1. Исследование входного проекта ⋯ 3

1.1–1.3. REST API, архитектура, сборка и запуск ⋯ 3

1.4–1.6. База данных, роли и инструменты ⋯ 4

1.7–1.9. Проверки и выявленные ошибки ⋯ 5

Глава 2. Разработка логики модуля ⋯ 6

2.1. Постановка задачи ⋯ 6

2.2. Словесный алгоритм ⋯ 6

2.3. Блок-схемы алгоритмов ⋯ 7

Глава 3. Программная реализация ⋯ 8

3.1–3.3. Конструкции C++, модуль и интеграция ⋯ 8

Глава 4. Тестирование и публикация ⋯ 9

4.1–4.5. Методика, пример, границы, ошибки и авторизация ⋯ 9

4.6. Сохранение в git и комплект сдачи ⋯ 10

Заключение ⋯ 11

Список использованных источников ⋯ 11

Приложение А. Листинг variant_mine.hpp ⋯ 12

Приложение Б. Листинг task.hpp ⋯ 14

Приложение В. Блок-схема расчётной части ⋯ 16

Приложение Г. Скриншоты тестирования ⋯ 17

Приложение Д. Команды запуска и материалы защиты ⋯ 20

ВВЕДЕНИЕ

Цель работы — разработать на C++17 расчётный модуль варианта 13 и встроить его в готовый REST-сервер rest-c. Модуль оценивает использование автопарка за день и вычисляет показатели расхода и стоимости топлива.

Для достижения цели изучены структура исходного проекта, маршруты и схема SQLite; реализованы разбор запроса, проверка ограничений, расчёт и сериализация результата; проведены модульные и интеграционные проверки. Работа выполнена на основе исходного репозитория mal-corp/rest-c, коммит fd0df312e07e7f7cd4b7d293ac7c6b7f14dd46a0.

ГЛАВА 1. ИССЛЕДОВАНИЕ ВХОДНОГО ПРОЕКТА

1.1. Основы REST API

Клиент обращается к ресурсам сервера по HTTP. Метод GET получает данные, POST создаёт сеанс или выполняет расчёт, PUT изменяет пароль, DELETE удаляет учётную запись. Тела запросов и ответов передаются в JSON. Bearer-токен связывает защищённый запрос с пользователем; права определяются его ролью.

1.2. Структура проекта и назначение модулей

Файл / каталог	**Назначение**
main.cpp	Инициализация, middleware, регистрация маршрутов
src/database.hpp	SQLite, параметризованные запросы, мьютекс
src/routes/auth.hpp	Авторизация, сеансы и RBAC
src/routes/public.hpp	Публичная история, счётчики, системные тесты
src/modules/example/stats.hpp	Демонстрационный расчёт статистики
src/modules/individual/	Индивидуальный вариант и маршрут
frontend/index.html	Веб-коллекция HTTP-запросов

1.3. Сборка и запуск проекта

Проверенная среда: macOS, AppleClang 17.0.0, C++17, SQLite 3.43.2, CMake 4.4.4. Сборка CMake и CTest завершились успешно. Сервер запускался из корня проекта, чтобы файл data/rest.db создавался по ожидаемому относительному пути.

```
cmake -S . -B build
cmake --build build -j 2
ctest --test-dir build --output-on-failure
./build/rest_server
```

ГЛАВА 1. БАЗА ДАННЫХ И ДОСТУП

1.4. Схема базы данных и роли пользователей

Таблица	Содержимое
users	id, username, password (соль и хеш), role
sessions	token, user_id, expires_at
works	id, user_id, datetime, input_json, result_json
logs	id, datetime, status, message

Сеансы привязаны к users через user_id. Успешный расчёт сохраняет исходный JSON и вычисленный результат в works. Индивидуальная история выбирает записи текущего пользователя. Демонстрационные и индивидуальные расчёты используют общую таблицу; история не разделяет их по виду модуля. При старте сервера таблица sessions очищается.

Роль user выполняет расчёты и управляет собственным паролем. admin просматривает пользователей и логи, запускает тесты и управляет собой и учётными записями user. owner обладает полными административными правами. Удаление owner1 запрещено.

1.5. Обзор эндпоинтов

Метод и путь после /api/v1.1	Доступ
POST /auth/register, /auth/login	Гость
GET /counters, /history; GET /health*	Гость
GET /auth/profile; POST /auth/refresh, /auth/logout	Авторизованный
POST /works/calc, /works/calc-individual	Авторизованный
GET /works/history, /works/history-individual	Авторизованный
GET /auth/users; DELETE /auth/users	admin / owner
PUT /auth/users/password	По целевой учётной записи
GET /admin/stats, /admin/run-tests	admin / owner
GET /admin/run-tests-individual	admin / owner

* /health расположен в корне, без префикса API. Новый маршрут run-tests-individual запускает 20 проверок варианта; исходный run-tests сохранён отдельно.

1.6. Инструменты тестирования

Веб-коллекция открывалась через локальный HTTP-сервер на порту 8093. После входа токен автоматически подставляется в заголовок Authorization. Контрольные запросы дополнительно проверены через curl и Python urllib. CTest запускает те же 20 проверок расчётного модуля без HTTP и базы данных.

ГЛАВА 1. РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ

1.7. Проверка корректности эндпоинтов

Интеграционный сценарий tests/http_tests.py выполнил 70 HTTP-проверок. Подтверждены успешный вход для трёх ролей, профиль, продление и завершение сеанса, регистрация и конфликт имени, расчёты обоих модулей, сохранение и чтение истории, доступ к административным маршрутам и обработка некорректных запросов. Все 70 проверок прошли.

1.8. Выявленные несоответствия и исправления

Анализ исходного кода показал: admin мог менять пароль owner и другого admin, а также удалять другого admin. Внесены проверки роли целевой записи: admin разрешено управлять собой и пользователями user. Запрос без токена к списку пользователей и административной статистике ранее давал 403; теперь возвращается 401, а 403 означает отсутствие прав у авторизованного пользователя.

Общий error_handler заменял подробные ошибки маршрута строкой Request failed. Теперь уже сформированное тело ошибки сохраняется. Смена пароля несуществующего пользователя теперь возвращает 404. Индивидуальный расчёт проверяет успешность INSERT: при отказе БД возвращается 500, а успешный ответ выдаётся после сохранения.

Шаблонный парсер использовал поиск подстрок и stod, превращал отсутствующие значения в нули и усекал дробные счётчики машин. Для варианта реализован строгий числовой JSON-контракт. Неверные данные отклоняются до вычисления показателей и записи в works.

Особенности исходного проекта сохранены: публичная история доступна без токена; демонстрационные и индивидуальные записи смешаны; failed_validations считает ошибки в works, хотя индивидуальные ошибки туда не записываются. Поэтому этот счётчик не отражает число отказов нового модуля. Это не препятствует проверке варианта, но ограничивает смысл общей статистики.

1.9. Запуск встроенных автотестов

Исходный /admin/run-tests вернул status=success, total_tests=20, passed=20, failed=0. Новый /admin/run-tests-individual вернул такие же итоговые значения для тестов автопарка. Результаты сохранены в tests/builtin_results.json и tests/fleet_results.json; подробный HTTP-протокол находится в tests/http_results.json.

ГЛАВА 2. РАЗРАБОТКА ЛОГИКИ МОДУЛЯ

2.1. Постановка задачи

Входные данные описывают один день эксплуатации. Требуется определить долю активных машин, средний расход топлива на 100 км и стоимость топлива на один километр общего пробега. Денежная единица результата совпадает с единицей цены топлива. Программа оценивает только топливо; амортизация, зарплата и ремонт в cost_per_km не входят.

Поле	**Тип**	**Ограничение**
total_vehicles (T)	int	T > 0
active_vehicles_day (A)	int	1 ≤ A ≤ T
total_fuel_consumed (F)	double	F > 0, литры
total_km_driven (D)	double	D > 0; D / A ≤ 1500 км
average_fuel_price (P)	double	P > 0, денежных единиц/литр

Формулы: U = (double) A / T × 100; C = F / D × 100; K = F × P / D. Выходные поля: utilization_rate_percentage, average_consumption_per_100km, cost_per_km. В tasks.txt между некоторыми множителями пропущены знаки умножения; смысл формул подтверждается контрольным примером.

2.2. Словесный алгоритм

1. Проверить Bearer-токен; при отсутствии действующего сеанса вернуть 401.

2. Прочитать плоский JSON-объект. Проверить синтаксис чисел, наличие пяти обязательных полей, уникальность ключей и отсутствие лишних данных.

3. Для счётчиков машин проверить целочисленность и представимость в int. Для вещественных значений проверить конечность и положительность.

4. Проверить A ≤ T и D / A ≤ 1500. При нарушении вернуть HTTP 400 с описанием.

5. Убедиться в представимости результатов в double, затем вычислить U, C и K.

6. Сформировать JSON, записать исходные данные и результат в works. При ошибке сохранения вернуть 500; иначе вернуть 200 с показателями.

Входной контракт намеренно строгий: неизвестные и повторные ключи отклоняются. Числовые записи 20, 20.0 и 2e1 допустимы для счётчика, если математическое значение целое и находится в диапазоне int. Строки, null, массивы и NaN не являются допустимыми числовыми значениями.

2.3. БЛОК-СХЕМА ОБРАБОТКИ

Использованы условные обозначения ГОСТ 19.701-90: терминатор, данные, процесс, решение и направленные линии потока. Расчётная часть отдельно приведена в приложении В.

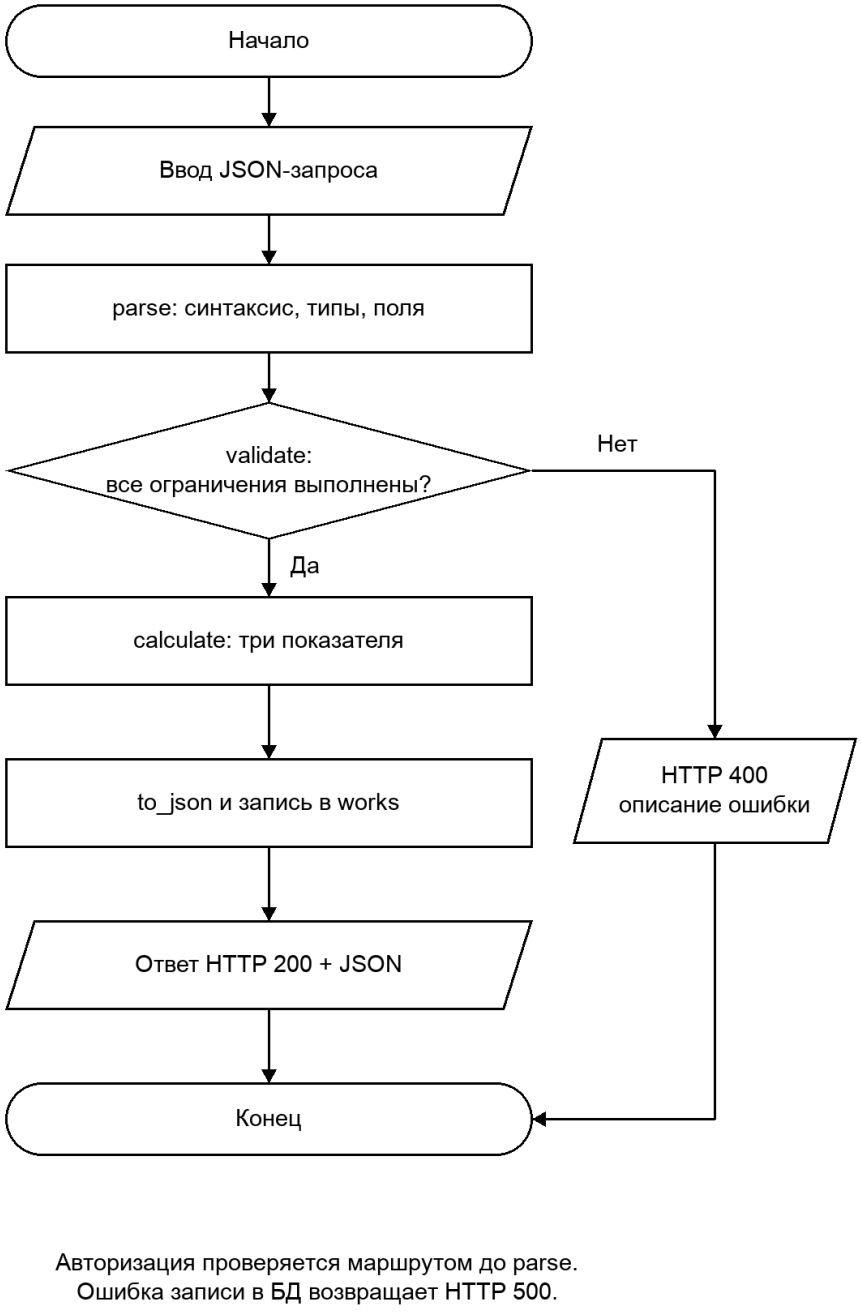


Рисунок 1 — Обработка запроса индивидуального модуля

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1. Выбор конструкций C++ и библиотек

Расчётный модуль использует только стандартную библиотеку C++17. Input и Output — структуры с именованными полями; std::map хранит разобранные ключи и позволяет обнаружить повторы; std::regex проверяет числовой синтаксис JSON; std::isfinite и std::trunc обеспечивают числовые проверки. Функции объявлены inline, поскольку реализация находится в заголовочном файле.

Разбор и вывод чисел используют locale::classic, чтобы десятичным разделителем была точка независимо от локали системы. Сериализация задаёт max_digits10 и сохраняет точность double. Поэтому 0,15 может отображаться как 0.15000000000000002: это особенность двоичного представления, а не ошибка формулы.

3.2. Реализация модуля варианта

Функция parse возвращает Input и заполняет parse_error при ошибке синтаксиса или типов. validate сначала возвращает эту ошибку, затем проверяет предметные ограничения. calculate вызывается маршрутом только после успешной валидации. to_json формирует объект с тремя полями результата.

При расчёте доли машин перед делением выполняется преобразование в double. Для расхода и стоимости используются промежуточные long double и порядок операций (F / D) × P, позволяющий избежать лишнего переполнения произведения F × P. Результаты вне диапазона double отклоняются с 400; деление на ноль исключено проверками F, D, T и A.

3.3. Встраивание в систему

В task.hpp подключён variant_mine.hpp вместо variant_roi.hpp. Пространство имён TaskVariant и интерфейс четырёх функций сохранены. Маршрут /works/calc-individual использует существующую авторизацию и таблицу works. Добавлены variant_tests.hpp и административный маршрут для запуска тестов варианта.

В frontend/index.html заменён пример тела индивидуального запроса на данные автопарка и добавлена кнопка «20 тестов автопарка». В CMakeLists.txt зарегистрирован исполняемый файл fleet_tests и проверка CTest fleet_variant_13. Полные листинги основного модуля и маршрута приведены в приложениях А и Б.

ГЛАВА 4. ТЕСТИРОВАНИЕ И ПУБЛИКАЦИЯ

4.1. Методика тестирования

Проверки проведены на трёх уровнях: функции модуля без сервера; HTTP-запросы к запущенному серверу; интерактивная работа в веб-коллекции. Ожидаемые результаты рассчитаны отдельно. Положительные сценарии проверяют значения, отрицательные — статус и наличие содержательной ошибки. После расчёта проверяется история текущего пользователя.

4.2. Контрольный пример и ручной расчёт

T=20; A=15; F=300 л; D=3000 км; P=1,5 денежных единиц/л. Средний пробег активной машины: 3000 / 15 = 200 км, что не превышает 1500 км.

U = 15 / 20 × 100 = 75%; C = 300 / 3000 × 100 = 10 л/100 км; стоимость топлива = 300 × 1,5 = 450; K = 450 / 3000 = 0,15 денежных единиц/км. Полученный JSON соответствует этим значениям с учётом погрешности double.

4.3. Граничные значения

Для T=1, A=1, F=1, D=1500, P=1: U=100%, C=1/15≈0,0666667 л/100 км, K=1/1500≈0,000666667. Ровно 1500 км на активную машину допускается. Для контрольных 15 машин допустимо D=22500; D=22500,1 отклоняется. При A=T загрузка равна 100%.

4.4. Невалидные данные

Проверены нулевые и отрицательные параметры, превышение числа активных машин, превышение суточного пробега, неполный объект, строки вместо чисел, дробный счётчик, переполнение int и double, повторные поля и повреждённый JSON. Отклонённые запросы возвращают 400 и не сохраняются как успешные расчёты.

4.5. Авторизация и результаты

Набор	**Результат**
Модуль варианта / CTest	20 из 20 проверок; CTest 1/1
Исходные встроенные тесты	20 из 20
HTTP-интеграция	70 из 70
Веб-коллекция	Вход, пример 200, ошибка 400, тесты 20/20

Без токена или с неверным токеном расчёт возвращает 401. Обычный user получает 403 на административные тесты. admin и owner запускают оба набора тестов. Попытка admin изменить пароль owner1 отклоняется с 403; после logout прежний токен не даёт получить профиль.

МАТРИЦА АВТОПРОВЕРОК ВАРИАНТА

№	Сценарий	Итог
1	Контрольный пример: три результата	Пройден
2	Минимальный автопарк и граница 1500 км	Пройден
3	Все машины активны	Пройден
4	Нулевое число машин	Пройден
5	Отрицательное число машин	Пройден
6	Нет активных машин	Пройден
7	Активных больше общего числа	Пройден
8	Нулевое топливо	Пройден
9	Отрицательное топливо	Пройден
10	Нулевой пробег	Пройден
11	Отрицательный пробег	Пройден
12	Больше 1500 км на машину	Пройден
13	Нулевая цена	Пройден
14	Отрицательная цена	Пройден
15	Не хватает полей	Пройден
16	Строка вместо числа	Пройден
17	Дробное число машин	Пройден
18	Выход за диапазон int	Пройден
19	Повреждённый JSON и дубли ключей	Пройден
20	Экспоненциальная запись и переполнение	Пройден

4.6. Сохранение в git и комплект сдачи

Изменения сохранены в локальном git-репозитории, коммит 7143530. Для сдачи выбран разрешённый инструкцией архив с кодом. Архив содержит исходники, тесты, результаты проверок, инструкции и git bundle с историей проекта. Удалённый публичный репозиторий не требуется для этого способа сдачи.

Чтобы восстановить git-историю из комплекта: git clone rest-c.bundle rest-c-git. Для обычного запуска достаточно распаковать каталог исходников и выполнить команды CMake из README_VARIANT_13.md. Бинарные файлы, база с сеансами и каталоги сборки в архив не включаются.

ЗАКЛЮЧЕНИЕ

Разработан и интегрирован модуль варианта 13. Реализованы пять входных параметров, проверка всех предметных ограничений, три расчётных показателя и JSON-контракт. Успешные результаты сохраняются в SQLite и доступны в истории пользователя.

Сборка выполнена через CMake. Подтверждено прохождение 20 проверок нового модуля, 20 исходных встроенных проверок и 70 HTTP-сценариев. Веб-интерфейс использует новый контрольный пример и позволяет запускать автопроверки. Исправлены выявленные нарушения административных прав и потеря подробного текста ошибок.

Подготовлены блок-схемы обработки и расчёта, листинги, снимки реальных проверок и инструкции для демонстрации. Результат работы может использоваться как учебный REST-сервис анализа дневной эксплуатации автопарка.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. mal-corp/rest-c. Исходный проект. https://github.com/mal-corp/rest-c (дата обращения: 29.10.2026).

2. Индивидуальное техническое задание: src/modules/individual/README.md исходного проекта.

3. Условия варианта 13: data/tasks.txt исходного проекта.

4. ГОСТ 19.701-90. ЕСПД. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. Карточка Росстандарта: https://protect.gost.ru/gost/details/0c85e066-f89e-433d-a6d3-b7f746bd0c7c

5. Протоколы выполненных проверок: tests/fleet_results.json, tests/builtin_results.json, tests/http_results.json в комплекте проекта.

ПРИЛОЖЕНИЕ А. ЛИСТИНГ МОДУЛЯ

src/modules/individual/variant_mine.hpp

```
#pragma once
// Вариант 13: анализатор утилизации и эффективности автопарка.
#include <string>
#include <map>
#include <regex>
#include <cmath>
#include <limits>
#include <sstream>
#include <iomanip>
#include <locale>


namespace TaskVariant {
struct Input {
    int total_vehicles = 0, active_vehicles_day = 0;
    double total_fuel_consumed = 0, total_km_driven = 0, average_fuel_price = 0;
    std::string parse_error;
};
struct Output {
    double utilization_rate_percentage, average_consumption_per_100km, cost_per_km;
};
// Строгий плоский JSON-контракт: пять уникальных числовых полей.
inline Input parse(const std::string& body) {
    Input in;
    size_t pos = 0;
    auto ws = [&] { while (pos < body.size() &&
        (body[pos]==' ' || body[pos]=='\t' || body[pos]=='\r' || body[pos]=='\n')) ++pos; };
    auto fail = [&](const std::string& e) { in.parse_error = e; return in; };
    ws();
    if (pos == body.size() || body[pos++] != '{') return fail("Expected JSON object");
    std::map<std::string, double> values;
    static const std::regex
field(R"json("([a-z_0-9]+)"\s*:\s*(-?(?:0|[1-9][0-9]*)(?:\.[0-9]+)?(?:[eE][+-]?[0-9]+)?))json");
    ws();
    if (pos < body.size() && body[pos] != '}') {
        while (true) {
            std::smatch m;
            const std::string tail = body.substr(pos);
            if (!std::regex_search(tail, m, field, std::regex_constants::match_continuous))
                return fail("Expected numeric field in JSON object");
            const std::string key = m[1];
            if (key != "total_vehicles" && key != "active_vehicles_day" &&
                key != "total_fuel_consumed" && key != "total_km_driven" && key !=
"average_fuel_price")
                return fail("Unknown field: " + key);
            if (values.count(key)) return fail("Duplicate field: " + key);
            std::istringstream number(m[2].str()); number.imbue(std::locale::classic());
            double value; number >> value;
            if (number.fail() || !std::isfinite(value)) return fail("Number out of range: " +
key);
            values[key] = value;
            pos += m.length(); ws();
            if (pos < body.size() && body[pos] == ',') { ++pos; ws(); continue; }
            break;
        }
    }
    if (pos == body.size() || body[pos++] != '}') return fail("Expected closing brace");
    ws(); if (pos != body.size()) return fail("Unexpected content after JSON object");
    for (const auto& key : {"total_vehicles", "active_vehicles_day", "total_fuel_consumed",
"total_km_driven", "average_fuel_price"})
        if (!values.count(key)) return fail(std::string("Missing field: ") + key);
    for (const auto& key : {"total_vehicles", "active_vehicles_day"}) {
        double v = values[key];
        if (v < 1 || v > std::numeric_limits<int>::max() || std::trunc(v) != v)
            return fail(std::string(key) + " must be a positive integer within int range");
    }
    in.total_vehicles = static_cast<int>(values["total_vehicles"]);
    in.active_vehicles_day = static_cast<int>(values["active_vehicles_day"]);
    in.total_fuel_consumed = values["total_fuel_consumed"];
    in.total_km_driven = values["total_km_driven"];
    in.average_fuel_price = values["average_fuel_price"];
    return in;
}
inline Output calculate(const Input& in) {
    return {static_cast<double>(in.active_vehicles_day) / in.total_vehicles * 100.0,
            static_cast<double>((static_cast<long double>(in.total_fuel_consumed) /
```

```
in.total_km_driven) * 100.0L),
            static_cast<double>((static_cast<long double>(in.total_fuel_consumed) /
in.total_km_driven) * in.average_fuel_price)};
}
inline std::string validate(const Input& in) {
    if (!in.parse_error.empty()) return in.parse_error;
    if (in.total_vehicles <= 0) return "total_vehicles must be > 0";
    if (in.active_vehicles_day < 1 || in.active_vehicles_day > in.total_vehicles)
        return "active_vehicles_day must be in [1, total_vehicles]";
    if (!std::isfinite(in.total_fuel_consumed) || in.total_fuel_consumed <= 0)
        return "total_fuel_consumed must be finite and > 0";
    if (!std::isfinite(in.total_km_driven) || in.total_km_driven <= 0)
        return "total_km_driven must be finite and > 0";
    if (in.total_km_driven / in.active_vehicles_day > 1500.0)
        return "total_km_driven / active_vehicles_day must be <= 1500 km";
    if (!std::isfinite(in.average_fuel_price) || in.average_fuel_price <= 0)
        return "average_fuel_price must be finite and > 0";
    // Проверка представимости без деления на ноль и переполнения double.
    const long double consumption = (static_cast<long double>(in.total_fuel_consumed) /
in.total_km_driven) * 100.0L;
    const long double cost = (static_cast<long double>(in.total_fuel_consumed) /
in.total_km_driven) * in.average_fuel_price;
    if (consumption > std::numeric_limits<double>::max() || cost >
std::numeric_limits<double>::max())
        return "Calculated result exceeds double range";
    return "";
}
inline std::string to_json(const Output& out) {
    std::ostringstream s; s.imbue(std::locale::classic());
    s << std::setprecision(std::numeric_limits<double>::max_digits10)
      << "{\"utilization_rate_percentage\":" << out.utilization_rate_percentage
      << ",\"average_consumption_per_100km\":" << out.average_consumption_per_100km
      << ",\"cost_per_km\":" << out.cost_per_km << "}";
    return s.str();
}
}
```

ПРИЛОЖЕНИЕ Б. ЛИСТИНГ МАРШРУТА

src/modules/individual/task.hpp

```
#pragma once
// ============================================================================
// МОДУЛЬ INDIVIDUAL: каркас роута.
// Не содержит математики — только вызовы функций варианта.
// ============================================================================

#include <string>
#include "../../lib/httplib.h"
#include "../../database.hpp"
#include "../../json.hpp"
#include "../../routes/auth.hpp"

// ─────────────────────────────────────────────────────────────────────────────
// ПОДКЛЮЧИТЕ СВОЙ ВАРИАНТ ЗДЕСЬ (замените одну строку).
// ─────────────────────────────────────────────────────────────────────────────
#include "variant_mine.hpp"
#include "variant_tests.hpp"

namespace Individual {
    using namespace std;
    using namespace httplib;

    inline void init_routes(Server& svr) {

        svr.Get("/api/v1.1/admin/run-tests-individual", [](const Request& req, Response& res) {
            string role, name;
            if (!Auth::auth(req, role, name)) { res.status = 401; return; }
            if (!Auth::admin(role)) { res.status = 403; return; }
            JArr cases; int passed = 0;
            const auto tests = FleetTests::run();
            for (size_t i = 0; i < tests.size(); ++i) {
                JObj item; item.int_("tc_id", i + 1); item.str("name", tests[i].first);
                item.str("status", tests[i].second ? "PASSED" : "FAILED");
                cases.add(item.str()); if (tests[i].second) ++passed;
            }
            JObj report; report.str("status", passed == (int)tests.size() ? "success" :
"failed");
            report.int_("total_tests", tests.size()); report.int_("passed", passed);
            report.int_("failed", tests.size()-passed); report.raw("test_cases", cases.str());
            res.set_content(report.str(), "application/json");
        });

        // ─── POST /works/calc-individual ──────────────────────────────────
        svr.Post("/api/v1.1/works/calc-individual", [](const Request& req, Response& res) {
            string role, name;
            int uid = Auth::auth(req, role, name);
            if (!uid) { res.status = 401; return; }

            auto in = TaskVariant::parse(req.body);

            string err = TaskVariant::validate(in);
            if (!err.empty()) {
                JObj o; o.str("error", err);
                res.status = 400;
                res.set_content(o.str(), "application/json");
                return;
            }

            auto out = TaskVariant::calculate(in);
            string result = TaskVariant::to_json(out);

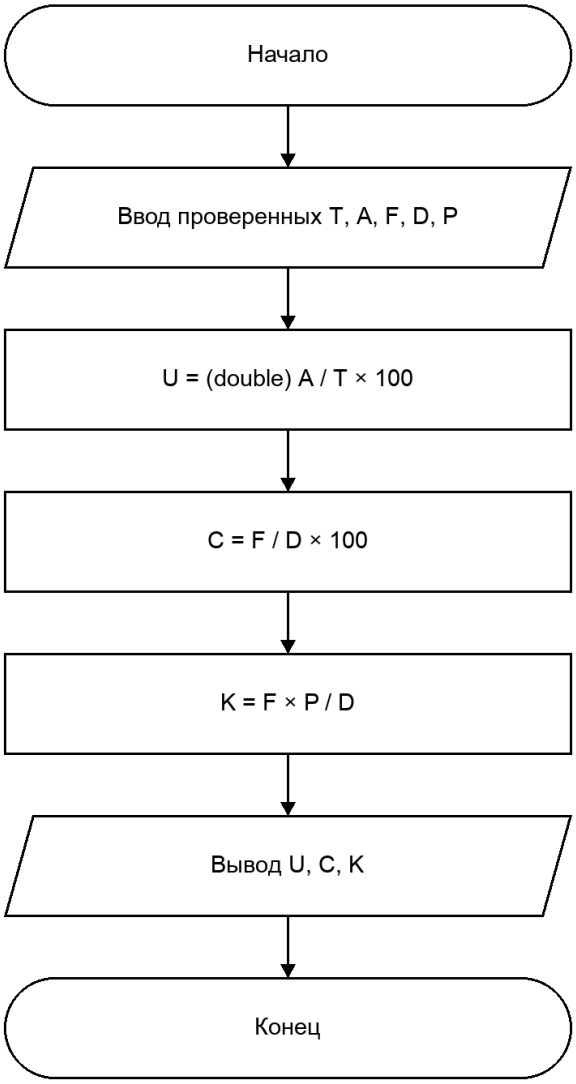
            if (!db_run("INSERT INTO works (user_id,datetime,input_json,result_json) "
                   "VALUES (?,?,?,?);",
                   {to_string(uid), now_iso(), req.body, result})) {
                res.status = 500;
                res.set_content("{\"error\":\"Could not save result\"}", "application/json");
                return;
            }
            res.set_content(result, "application/json");
        });
```

```
        // ─── GET /works/history-individual ──────────────────────────────────
        svr.Get("/api/v1.1/works/history-individual", [](const Request& req, Response& res) {
            string role, name;
            int uid = Auth::auth(req, role, name);
            if (!uid) { res.status = 401; return; }

            int limit = MAX_HISTORY_LIMIT;
            if (req.has_param("limit")) {
                try { limit = min(stoi(req.get_param_value("limit")), limit); }
                catch (...) {}
            }

            JArr arr;
            for (auto& r : db_all(
                "SELECT id,datetime,input_json,result_json FROM works "
                "WHERE user_id=" + to_string(uid) +
                " ORDER BY id DESC LIMIT " + to_string(limit) + ";")) {
                JObj o;
                o.int_("id", stoll(r[0]));
                o.str("datetime", r[1]);
                o.raw("input", r[2]);
                o.raw("result", r[3]);
                arr.add(o.str());
            }
            res.set_content(arr.str(), "application/json");
        });
    }
}
```

ПРИЛОЖЕНИЕ В. РАСЧЁТНАЯ БЛОК-СХЕМА



T — всего машин; A — активных; F — топливо;
D — пробег; P — цена; U — загрузка;
C — расход на 100 км; K — стоимость км.

Рисунок 2 — Алгоритм расчёта показателей автопарка

ПРИЛОЖЕНИЕ Г. КОНТРОЛЬНЫЙ ПРИМЕР

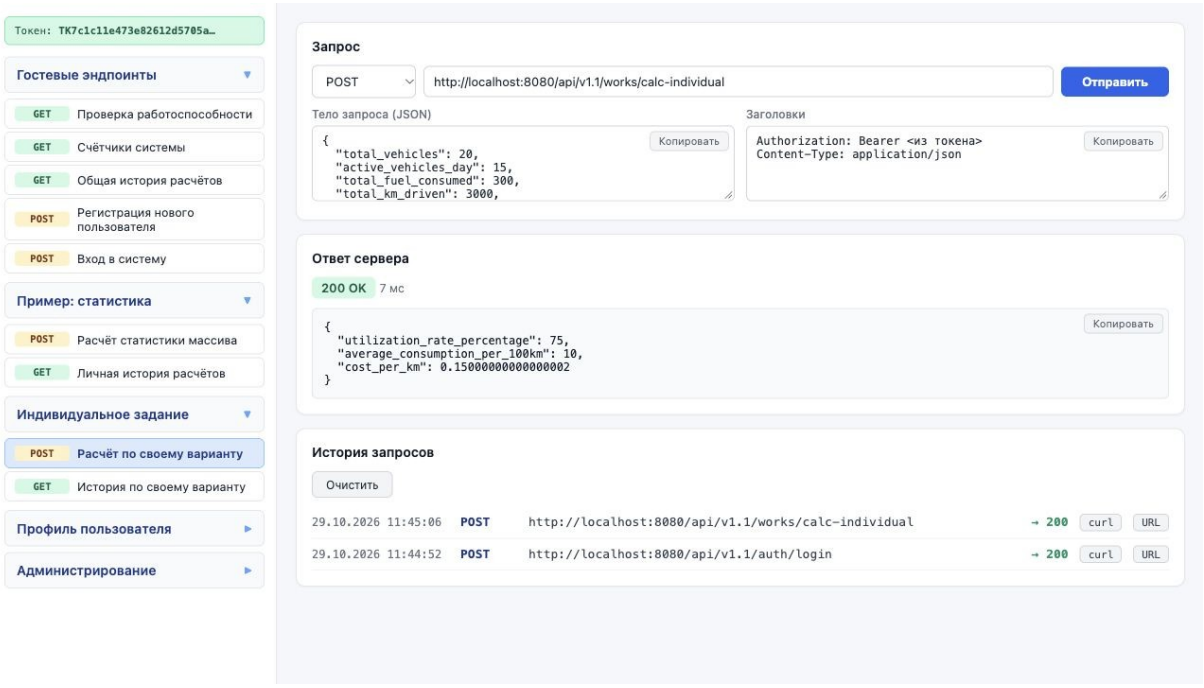


Рисунок 3 — Реальный ответ HTTP 200 в веб-коллекции

ПРИЛОЖЕНИЕ Г. ОШИБКА ВАЛИДАЦИИ

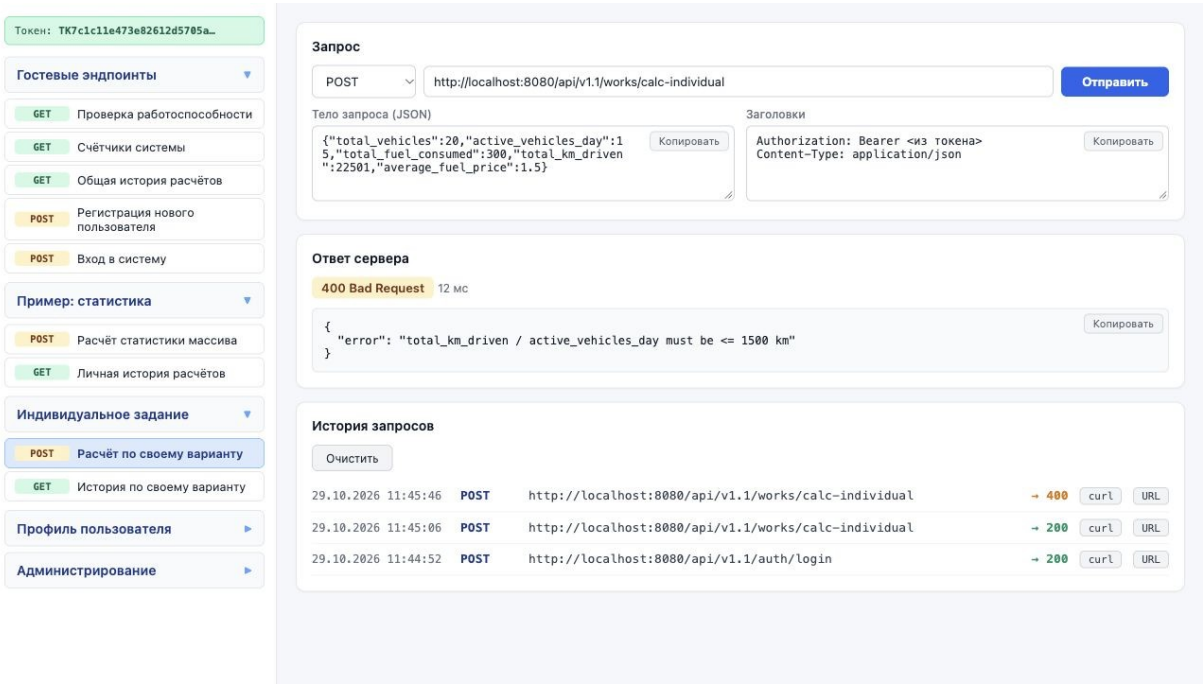


Рисунок 4 — Отказ при пробеге выше 1500 км на машину

ПРИЛОЖЕНИЕ Г. АВТОПРОВЕРКИ

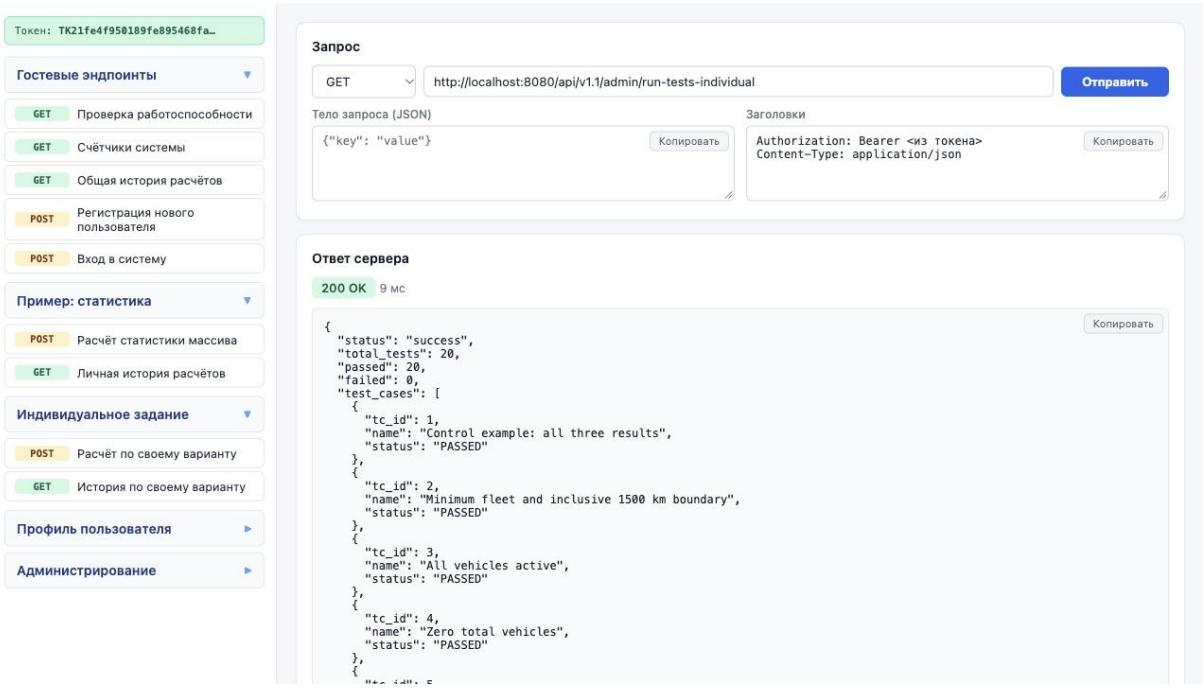


Рисунок 5 — Административный запуск 20 проверок

ПРИЛОЖЕНИЕ Д. ДЕМОНСТРАЦИЯ И ЗАЩИТА

Порядок демонстрации

1. Собрать проект через CMake и выполнить CTest.

2. Запустить сервер; открыть /health и веб-коллекцию frontend/index.html.

3. Войти как user1 / user1, отправить контрольный пример варианта 13 и пояснить три результата.

4. Изменить пробег на 22501: показать 400. Открыть историю и найти сохранённый корректный расчёт.

5. Войти как admin1 / admin1 и выполнить оба набора автотестов.

6. Показать структуры Input, Output и четыре функции; объяснить проверку ограничений до расчёта.

Краткие ответы на вопросы

Почему нужен double перед A/T? Без преобразования деление двух int даст целую часть: 15/20=0. Почему делитель D не равен нулю? Это запрещено validate до вызова calculate. Почему проверяем D/A? Условие ограничивает средний дневной пробег одной активной машины.

Где хранится результат? В works вместе с user_id и исходным JSON. Зачем проверять JSON отдельно от формул? Правильная формула не защищает от пропущенного поля, строки вместо числа или переполнения. Чем отличаются 401 и 403? Первое — нет действующего сеанса, второе — сеанс есть, но роли недостаточно.

Пример запроса curl

```
curl -X POST http://localhost:8080/api/v1.1/works/calc-individual \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"total_vehicles":20,"active_vehicles_day":15,
       "total_fuel_consumed":300,"total_km_driven":3000,
       "average_fuel_price":1.5}'
```